Waybill — a courier routing engine that proves what it claims

The assignment says TSP, Greedy, and Branch & Bound. Couriers do not solve TSP. This is the plan for building what they actually solve, on real roads — and for never claiming more about an answer than the evidence supports.

TRUE PROBLEM	MANDATORY SOLVERS	DISTANCES	VALIDATION	PHASES
CVRPTW	6 + 2 optional	OSRM via adapter	4-link chain	9, gated

What changed in revision 2

Response to the architecture review. The review found one real contradiction and several places where revision 1 claimed more than it could support.

ACCEPTED — THE REVIEW WAS RIGHT

- **The purity contradiction.** R1 said `core/` imports no network code, then put the OSRM client inside `core/distance/`. Fixed with an `adapters/` layer.
- **Pre-claimed solver scale.** R1 asserted "1-tree makes B&B viable past n=40" and supplied a template sentence with invented numbers. Both removed.
- **Conflated claims.** A proven-optimal TSP tour said nothing about whether a 150-stop CVRPTW plan is good. Now separate, typed statuses.
- **Loose use of "optimality gap."** Only valid against a proven optimum or valid bound.

REFINED — ACCEPTED WITH CHANGES

- **Solver count.** Agreed on tiering, but the 1-tree bound moves to *recommended*, not stretch — gated on the bound-validity invariant rather than deferred.
- **Metaheuristic choice.** The review left ALNS-or-SA open. This plan picks **simulated annealing**, for reasons stated in §05.
- **Capacity dimension.** One dimension for the baseline — parcel count, not weight, because violations are then visible by inspection.

ADDED — IN NEITHER DOCUMENT

- **The asymmetry trap (§07).** Textbook 2-opt is *invalid* on the asymmetric matrices this project deliberately introduces. This changes which operators are mandatory.
- **Validation continuity (§08).** Both documents left an unchecked band between brute force and TSPLIB. Now closed explicitly.
- **The confidently-wrong failure mode (§06).** An invalid lower bound does not crash — it silently prunes the optimum and reports success.

00 In plain terms KEPT

No jargon. If you can explain the project this way, you can defend it in a viva.

What is the actual job?

A delivery company has a warehouse and 150 parcels to deliver today. It has 6 vans. Somebody has to decide two things: **which van takes which parcels**, and **what order each driver visits their stops in**.

Get it right and every driver is home by 5pm having burned less fuel. Get it wrong and one van is overloaded, another is half empty, and somebody is still driving at 8pm. That decision is made every morning by every delivery company on earth.

Why can't a computer just try every option?

Because the number of options doesn't grow steadily — it explodes.

10 STOPS	15 STOPS	20 STOPS
181,440	**43.6 billion**	**60.8 quadrillion**
possible routes. A laptop checks all of them before you blink.	possible routes. Now about 44 seconds of solid computing.	possible routes. Checking a billion every second: **1.9 years**.

Twenty stops is a light morning for one driver. So "just check everything" stops working almost immediately — and no faster computer rescues you, because five more stops multiplies the work by millions again.

So what do you do instead?

The quick way — be greedy. Repeatedly drive to the nearest stop you haven't visited. Like being at a party and always talking to whoever is standing closest. Instant, reasonable — but because you never planned ahead you end up stranded across the room, facing one long miserable drive home.

The careful way — be clever about what you skip. Suppose you've found a route costing 87 km. Now you look at a huge group of routes that all begin with the same bad first move, and you can prove that *even in the luckiest case* anything in that group costs at least 94. You throw the entire group away without checking one of them.

That is **Branch & Bound**. You're not searching faster — you're proving that enormous regions can't contain the winner.

What makes this project different?

Most student projects implement one method and print a route. The route looks plausible, and nobody can tell whether it's good, because there's nothing to compare it to.

This project builds both kinds, runs them on identical problems, and measures the difference — then **labels every number with how much it is actually entitled to claim**. "Proven best" and "best we found" are different statements, and the system never confuses them.

Why does the map part matter?

Most projects measure distance in a straight line, as if vans could fly. Two addresses might be 2 km apart as the crow flies but a 3.4 km drive, because of one-way streets and a river. Worse, driving A→B is often a *different distance* than B→A.

That last detail is not cosmetic. It quietly breaks one of the standard textbook algorithms, which is the subject of §07 and one of the more interesting findings available here.

THE SIX WORDS WORTH KNOWING

TSP

Travelling Salesman Problem. One vehicle, a list of stops: what's the shortest round trip visiting each once? The hard problem this is built on.

GREEDY

Any method that always takes the choice that looks best right now. Fast, simple, usually decent — never guaranteed.

BRANCH & BOUND

Split the possibilities into groups, calculate the best conceivable outcome for each group, and discard whole groups whose best case is still worse than an answer you already hold.

LOWER BOUND

A number you can prove no route in a group can beat. The engine of Branch & Bound — and if it's ever wrong in the optimistic direction, the whole proof collapses silently.

OPTIMALITY GAP

How much worse a fast answer is than a *proven* best one, as a percentage. If you don't have a proven best, you don't have this number — you have something weaker.

CVRPTW

The realistic version: multiple vehicles, capacity limits, and delivery time windows. Ugly name, but it's the problem actual couriers have.

---

01 The problem the title gets wrong KEPT

Unchanged from revision 1. The review endorsed this framing.

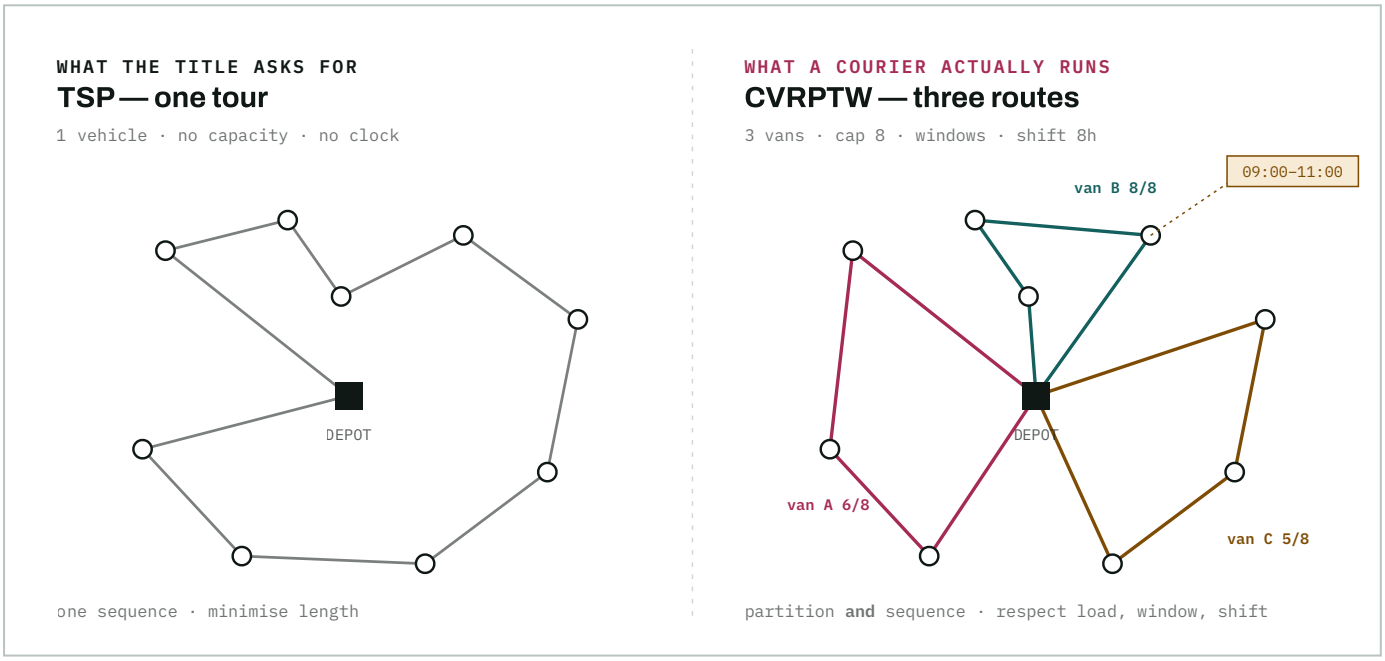
> TSP is the kernel. It is not the system. Building the kernel and calling it a courier system is the single reason these projects read as coursework.

The **Travelling Salesman Problem** asks for the minimum-cost Hamiltonian cycle over `n` nodes. It is NP-hard. A symmetric instance has `(n−1)!/2` distinct tours, so `n = 20` is already about $6{\times}10^{16}$ of them.

But a courier fleet violates every simplifying assumption TSP makes:

- **Multiple vehicles.** Deciding which van takes which stops is a partitioning problem sitting on top of the sequencing problem — and it dominates the cost.
- **Capacity.** A tour optimal in distance is worthless if it needs 1.4 vans.
- **Time windows.** "Between 09:00 and 11:00" turns ordering into scheduling. Early means waiting; late means failing.
- **Service time.** Minutes at the door — often larger than driving time between adjacent urban stops.
- **Asymmetry.** On a real road network `d(A,B) ≠ d(B,A)`. Every textbook symmetric-matrix assumption breaks — see §07 for what that actually costs you.
- **Shift limits.** Legal maximum hours and mandated breaks.

The correct name is CVRPTW. TSP is the special case with one vehicle, infinite capacity, and no clock.



Identical nine stops, two different problems. The left panel optimises a sequence. The right must first decide which van gets which stops, then sequence each, then check every arrival against a clock and every load against a limit. The partitioning decision on the right is where most of the cost — and all of the difficulty — lives.

THE FRAMING THAT WINS

Implement TSP with Greedy and Branch & Bound exactly as specified, prove they work — then show, with your own measurements, that the TSP formulation cannot express a real delivery day, and extend it. You answer the assignment *and* critique it with evidence.

02 The claim model NEW · P0

The review's most important structural addition. Exactness is a property of one formulation, instance, and solver run — never of the application.

> Revision 1 let a proven-optimal 12-stop TSP tour sit in the same system as a heuristic 150-stop fleet plan, with nothing in the types to stop the word "optimal" migrating from one to the other. That is the most dangerous kind of bug: it never crashes.

Every solve returns an explicit status. The UI, the report, and the benchmark tables all read that status and are forbidden from wording a result more strongly than it permits.

STATUS	MEANS	PERMITTED WORDING
PROVEN_OPTIMAL_TSP	An exact solver finished with a valid proof condition on a single-vehicle instance.	"Optimal" — for that TSP instance only.
EXACT_TIMEOUT_WITH_INCUMBENT	Exact search found feasible tours but hit its budget before proving optimality.	Report incumbent *and* best lower bound. Never "optimal".
HEURISTIC_FEASIBLE_CVRPTW	A multi-vehicle plan passed the independent feasibility gate.	"Feasible." Quality only ever relative to a stated comparator.
HEURISTIC_INFEASIBLE	Candidate violates capacity, window, shift, or visit-once constraints.	Reject. Never surface as a route.
REFERENCE_GAP_ONLY	Compared against OR-Tools or a best-known value, not a proven optimum.	"Gap to reference" — never "optimality gap".

WORDING RULE

Optimality gap is mathematically meaningful only when the denominator is a proven optimum or a valid bound. Against TSPLIB published optima it is legitimate. Against OR-Tools it is **not** — OR-Tools is a strong heuristic, not an oracle. Call that one gap to reference, and name the comparator every time it appears.

The SolveReport contract

Every solver returns one of these, and the benchmark harness refuses to plot a run that cannot fill in the mandatory fields.

```
SolveReport
├── problem_type        # TSP | CVRP | CVRPTW
├── status              # the enum above — mandatory
├── objective_value     # integer, in stated units
├── objective_units     # metres | seconds — never "cost"
├── best_lower_bound    # when an exact search exposes one
├── proven_gap           # (incumbent − lower_bound)/incumbent — only when valid
├── reference_value     # OR-Tools / TSPLIB comparator
├── reference_source    # provenance string — mandatory if reference_value set
├── stop_reason          # optimal | timeout | memory_cap | infeasible | cancelled
├── wall_time_ms, cpu_time_ms, peak_memory_mb
├── nodes_created, nodes_explored, nodes_pruned, peak_frontier   # B&B
└── seed, solver_config_hash, distance_matrix_hash, hardware_id
```

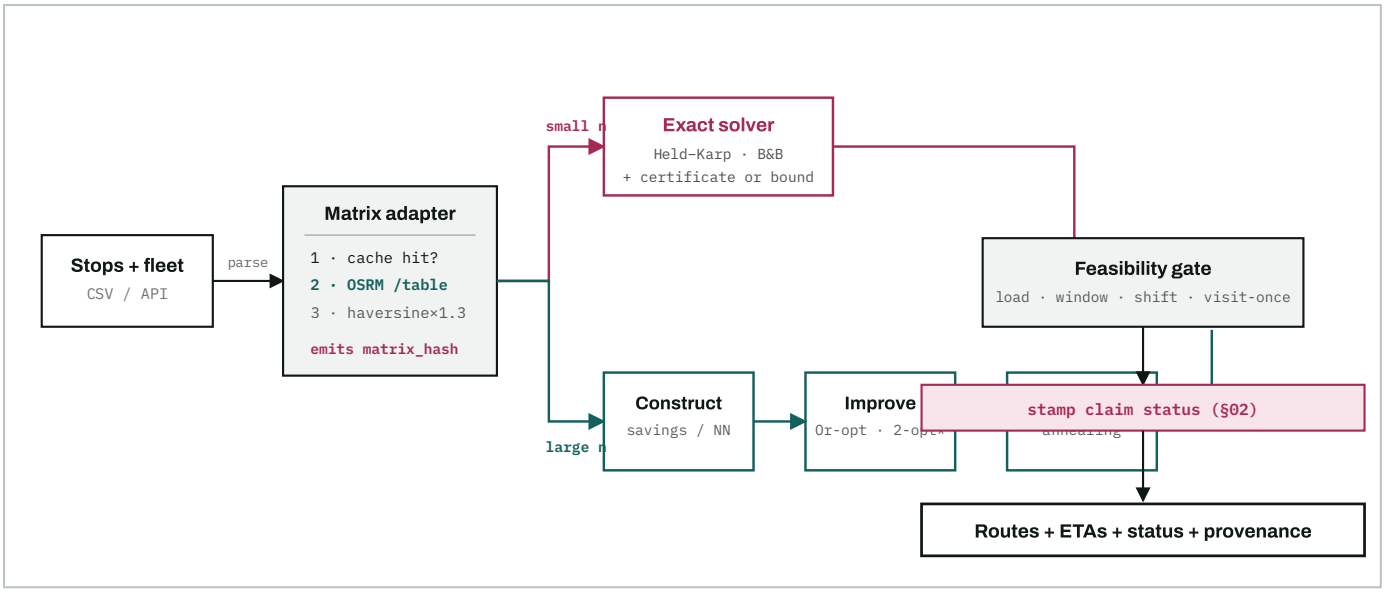
Two fields carry more weight than they look. `distance_matrix_hash` mechanically enforces that every compared solver saw the identical input — a comparison across two different matrices is meaningless, and the harness can now refuse it rather than trusting you to remember. And `stop_reason` is what makes exact-versus-heuristic comparison fair at all: a run that hit a timeout is not a data point about algorithm quality, it is a data point about the budget.

---

03 How the system works, end to end REVISED

Same pipeline as revision 1, with the claim status now flowing through it as a first-class output.

A solve is a pipeline. Addresses become coordinates, coordinates become a cost matrix, the matrix feeds a solver, and the solver's output is checked against every constraint before it is allowed out. Below a size threshold the system returns a provably optimal answer *with its certificate*; above it, a feasible answer labelled with what it is entitled to claim.



One pipeline, two lanes, one claim stamp. Both lanes pass the same independent feasibility gate, so a heuristic can never return a plan that quietly overloads a van. The stamp after the gate is what revision 1 was missing: nothing leaves the system without a machine-readable statement of what it is entitled to claim. Note the `2-opt*` — see §07 for why that asterisk exists.

---

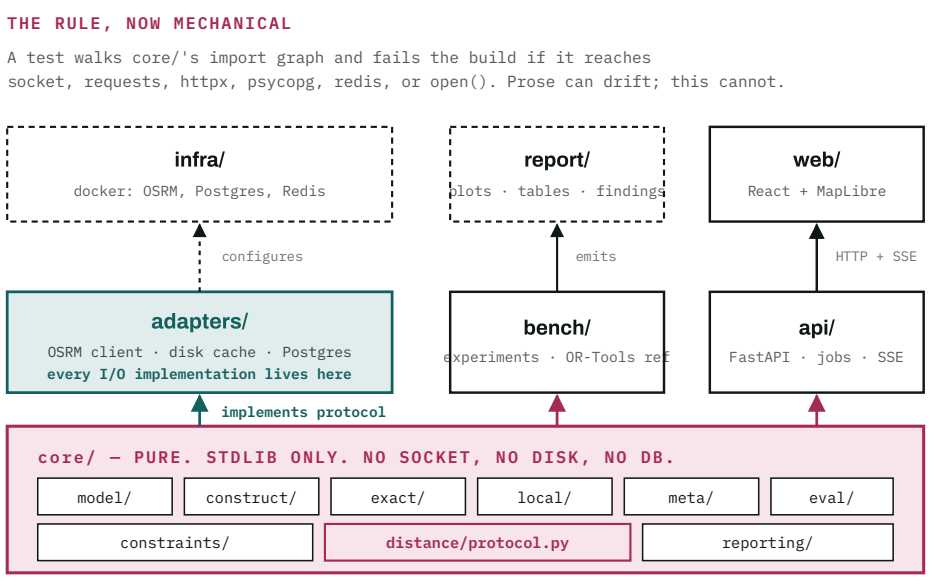
04 Architecture — make the purity rule mechanical

REVISED · P0

The review's best catch. Revision 1 asserted a rule its own repository layout broke.

> R1 said "`core/` imports nothing from the web, the database, or the network," then listed `core/distance/ — Protocol + haversine, OSRM, cached impls`. An OSRM client is a network client. The rule was prose; the layout contradicted it.

The fix is structural, not editorial. `core/` holds the *protocol* — what a distance matrix is and what a provider must offer. Every implementation that touches a socket, a disk, or a database lives in `adapters/`. Now the rule is enforced by the import graph and can be checked by a test, rather than by remembering.



The protocol stays in the core; every implementation that touches the outside world moves out. Because `bench/` and `api/` import the same pure solver code, a benchmark number is a claim about the deployed system. And because `core/` can reach nothing external, the entire solver suite runs offline in unit tests — which is what makes the validation chain in §08 cheap enough to run on every commit.

Repository layout

```
waybill/
├── core/                  # stdlib only — enforced by an import test
│   ├── model/               # Instance, Solution, Vehicle, Stop, TimeWindow
│   ├── distance/
│   │   ├── protocol.py      # DistanceMatrix / DistanceProvider interfaces  ← interface only
│   │   └── matrix.py        # validation, symmetry check, hashing
│   ├── construct/           # nearest_neighbour, savings, insertion
│   ├── exact/               # brute_force, held_karp, branch_bound/
│   │   └── bounds/          # reduction, hungarian, one_tree
│   ├── local/               # or_opt, relocate, swap, two_opt (symmetric-guarded)
│   ├── meta/                # simulated_annealing
│   ├── constraints/         # capacity, windows, shift, forward slack
│   ├── eval/                # objective + independent feasibility checker
│   └── reporting/           # SolveReport schema + status enum
├── adapters/              ← every socket, file, and DB call lives here
│   ├── distance/            # haversine, osrm_client, persistent_cache
│   └── storage/             # Postgres / SQLite result persistence
├── bench/                 # experiment runner, manifests, plots, OR-Tools ref
├── api/                   # thin FastAPI orchestration — no algorithms
├── web/                   # React + MapLibre
├── infra/                 # docker compose: osrm, postgres, redis
└── report/                # generated tables, figures, written analysis
```

WRITE THIS TEST IN PHASE 0

A dozen lines walking `core/`'s AST for imports of `socket`, `requests`, `httpx`, `psycopg`, `redis`, or bare `open()`, failing the build on a hit. It is the cheapest architectural guarantee in the project and it is the reason revision 1's contradiction could not recur.

---

05 Solver scope — fewer mandatory, more evidence REVISED

Nine partially-tested solvers are worth less than six fully validated ones. Tiering accepted from the review, with two changes.

METHOD	CLASS	TIER	PURPOSE
Brute force	EXACT	MANDATORY	Ground truth for n ≤ 12; the root of the validation chain
Held–Karp DP	EXACT	MANDATORY	Exact reference to the memory wall; exposes that wall as a result
B&B + reduced-cost bound	EXACT	MANDATORY	The assignment's named technique; baseline for bound comparison
B&B + Hungarian bound	EXACT	MANDATORY	Second bound — without it there is no bound-quality experiment
B&B + 1-tree / subgradient	EXACT	RECOMMENDED	A structurally different relaxation; gated on the validity invariant
Nearest neighbour	HEUR	MANDATORY	Greedy baseline — the assignment's "Greedy"
Clarke–Wright savings	HEUR	MANDATORY	Fleet-aware construction; answers partitioning and sequencing at once
Or-opt / relocate	HEUR	MANDATORY	Local search that stays valid under asymmetry (§07)
2-opt (symmetric-guarded)	HEUR	MANDATORY	Classic improvement — with the guard that makes it honest
Simulated annealing	HEUR	RECOMMENDED	One well-instrumented metaheuristic, for the anytime curve
OR-Tools	RULER	REFERENCE	External baseline in `bench/` only. Never inside a submitted solver.

Two departures from the review

The 1-tree bound moves from stretch to recommended. The review's concern is sound — a subtly wrong bound is worse than no bound, because it silently prunes the optimum. But the mitigation already exists in the review's own invariant list: *a lower bound must never exceed the true optimum on a brute-force-verifiable instance*. With that test running on a thousand small instances, an invalid 1-tree implementation

cannot reach the report. Gate it on the test rather than deferring it, because bound quality is the one place where this project can produce a genuinely interesting algorithmic result, and a comparison of two bounds from the same family (both LP-relaxation flavoured) is much less interesting than one that includes a spanning-tree relaxation.

Pick simulated annealing over ALNS. The review left this open; leaving it open is the risk. SA is roughly 80 lines with three parameters, its acceptance behaviour follows directly from the cooling schedule, and you can derive and defend that behaviour on a whiteboard. ALNS is several hundred lines of destroy/repair operators with adaptive weights — more impressive when it works, much harder to validate, and nearly impossible to explain rigorously under questioning. For a graded project where every line must be defensible, SA is the better risk-adjusted choice. If SA is finished, instrumented, and the anytime curve is generated with time to spare, then ALNS becomes a genuine stretch goal.

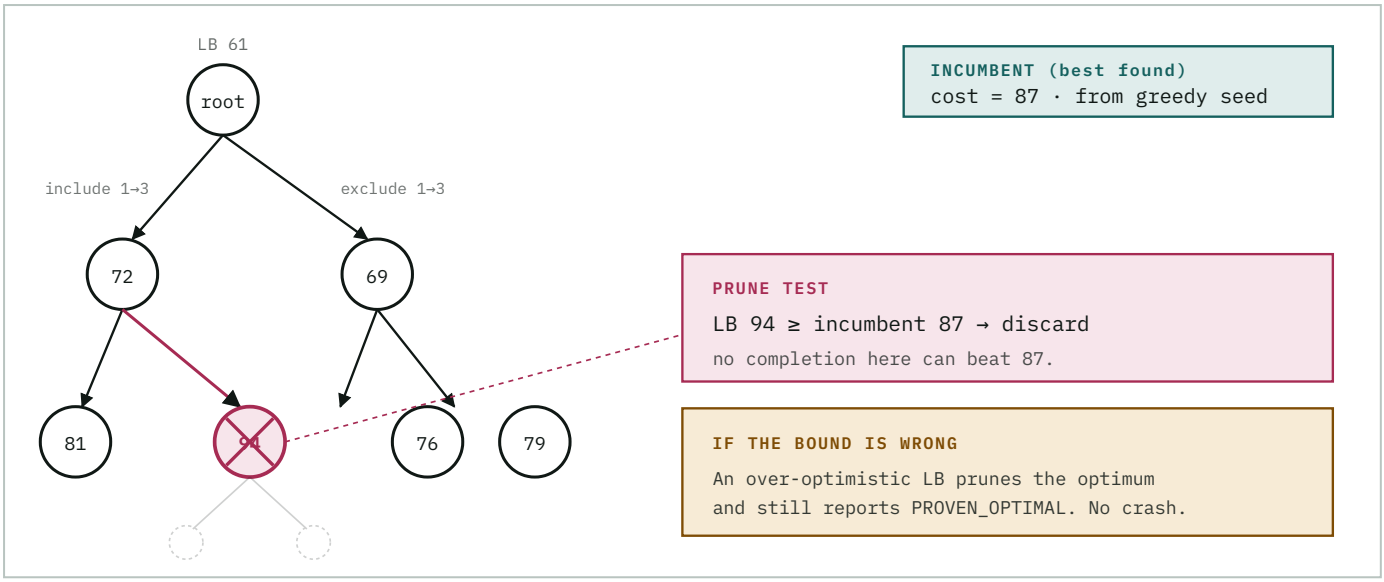
> SCOPE RULE
>
> A solver enters the final comparison only after it passes the feasibility checker, agrees with exact cross-validation where applicable, reproduces under a fixed seed, and fills every mandatory `SolveReport` field. **"Implemented" is not "included."** A method that fails any of these is reported as attempted-and-excluded, with the reason — which is itself a more honest result than quietly dropping it.

---

06 Branch & Bound — the experimental design REVISED

> Branch & Bound is a bound with a search wrapped around it. Revision 1 said that and then undercut itself by asserting how far the bounds would scale. The scale is the experiment's output, not its premise.

The mechanism is unchanged: maintain the best complete tour found (the **incumbent**); for each partial tour compute a **lower bound** on the best possible completion; if that bound is already ≥ the incumbent, discard the whole subtree unexplored.



One prune eliminates a subtree — and one invalid bound eliminates the answer. The amber panel is the failure mode neither the original plan nor the review's tables made vivid: a bound that is too high does not error, it quietly discards the branch containing the optimum and returns a confident wrong answer. This is why the bound-validity invariant in §09 is the single most important test in the project.

Control variables — so differences are attributable to the bound

The bound-quality experiment only means something if the bound is the *only* thing that changes. Hold constant: the instance, the incumbent initialisation (same greedy-plus-local-search seed tour for every variant), the branching rule, the search strategy, and the timeout.

- **Record bound computation time separately.** A tighter bound that costs too much per node can lose overall — that trade-off is a result, and you can only see it if the two costs are measured apart.
- **Record peak frontier size.** Best-first's memory blow-up then becomes a measured curve instead of a crash.
- **On timeout, save the incumbent and the best remaining lower bound.** The run still yields a certified residual gap — a timeout becomes data rather than a hole in the table.
- **Expose DFS and best-first, but fix one as the default** for the primary bound experiment, or you are varying two things at once.

CLAIMS DELETED FROM REVISION 1

These were written as expectations and read as findings. They are hypotheses drawn from the literature, and the submitted report must replace them with measured ranges from your implementation on your hardware:

"1-tree makes B&B viable past n = 40." · "ALNS lands within 1%." · "Nearest neighbour is ~25% worse." · "Forward slack gives two orders of magnitude." · And the entire worked "sentence to aim for," which supplied invented numbers in a shape designed to be copied.

What replaces them is a sentence with blanks, filled only after the experiments run: *"On this hardware and instance family, exact method X remained provably optimal through n = ___ under a ___-second budget; beyond that, heuristic Y achieved a median gap of ___% in ___ ms. Changing only the lower bound reduced explored nodes by ___×."*

---

07 The asymmetry trap NEW

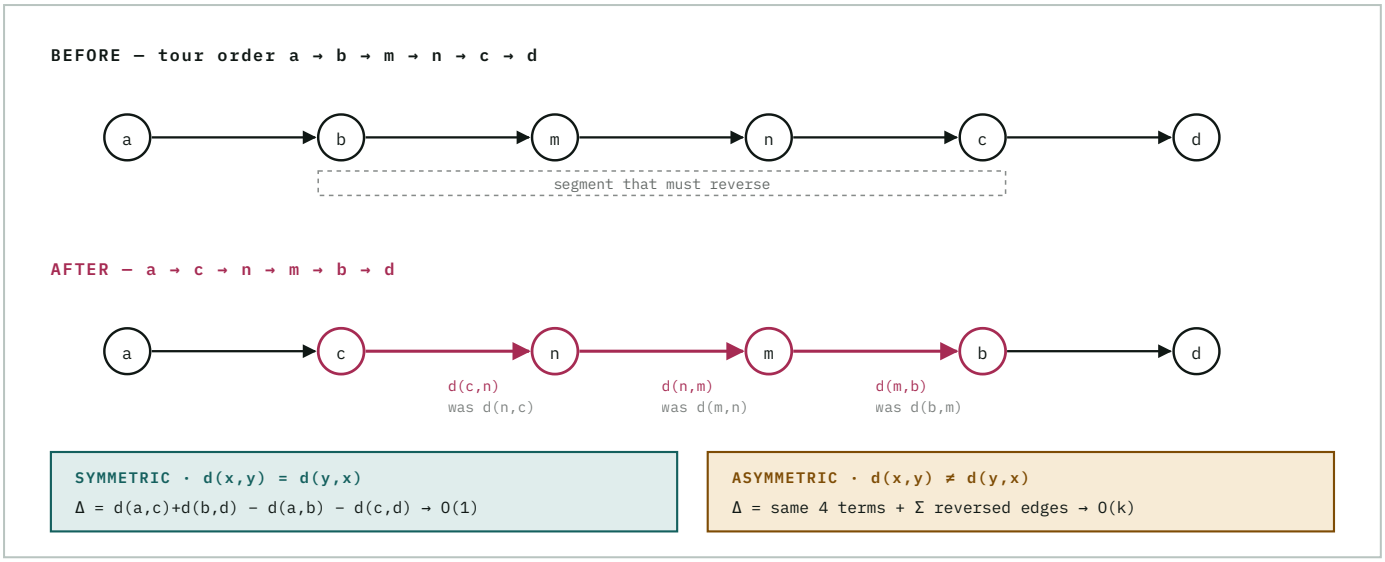
Absent from both revision 1 and the review. It connects the "real roads" feature directly to algorithm correctness, and it changes which operators are mandatory.

> Textbook 2-opt is not valid on an asymmetric matrix. Since this project deliberately introduces asymmetric road distances, that is a correctness problem sitting directly between two of its headline features.

2-opt removes two edges from a tour and reconnects them the other way. Remove `(a,b)` and `(c,d)`, add `(a,c)` and `(b,d)` — and to keep the tour connected, the entire path from `b` to `c` must be **traversed in reverse**.

On a symmetric matrix that reversal is free: every internal edge `(x,y)` becomes `(y,x)`, and `d(x,y) = d(y,x)`, so the internal cost is unchanged. The move's delta collapses to four terms and evaluates in constant time. That `O(1)` delta is the entire reason 2-opt is fast enough to be useful.

On an asymmetric matrix `d(x,y) ≠ d(y,x)`, so every reversed internal edge changes cost. The delta now includes a sum over the whole reversed segment: `O(k)` for a segment of length `k`. Evaluate the neighbourhood naively and 2-opt silently becomes an order of magnitude slower — or, far worse, keep the four-term formula and the search is now optimising a cost function that *is not the tour cost*.



The reversal is free only when the matrix is symmetric. Under OSRM road distances every accent edge changes cost when the segment flips, so the four-term delta is simply the wrong number. Using it does not raise an error — the search happily optimises a quantity that is not the tour length.

What this changes in the plan

- **Or-opt and relocate become mandatory, not optional.** They move a short segment without reversing it, so their delta stays `O(1)` and correct under asymmetry. This is the asymmetry-safe workhorse — and it is why the review's instinct to require "one relocation operator" was right, though for a reason it did not state.
- **2-opt gets a symmetry guard.** The matrix carries an `is_symmetric` flag, set once at construction. 2-opt asserts on it, or switches to the honest `O(k)` delta. It must never silently use the four-term formula on an asymmetric input.
- **The TSP experiments run on symmetric instances** (TSPLIB is symmetric, and random Euclidean is symmetric), so the classical comparison is clean and comparable to published results. The **asymmetric road matrix belongs to the CVRPTW demo and the road-matrix experiment**, where Or-opt-family operators do the improving.

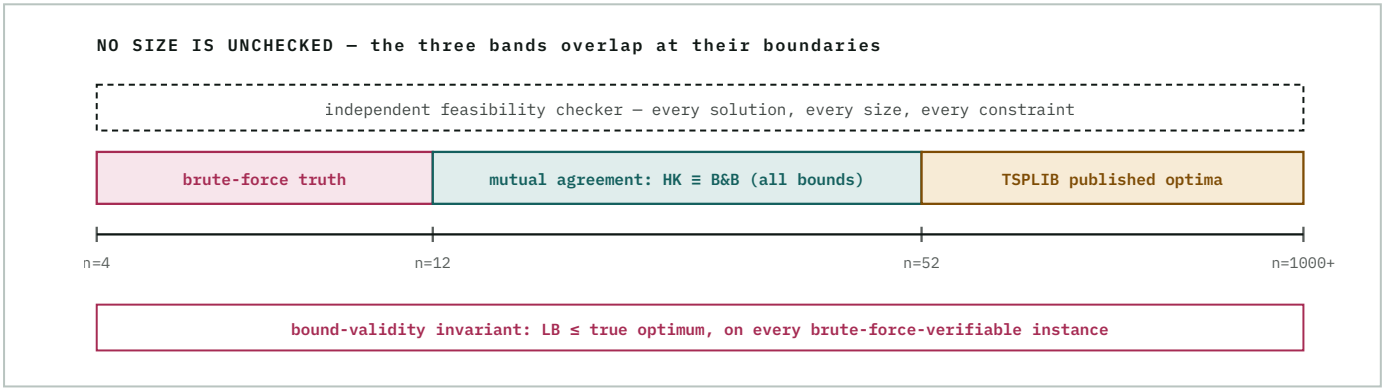
> THIS IS A FREE FINDING
>
> Take one instance. Solve it with the symmetric Haversine matrix, then with the OSRM road matrix over the identical stops. Report how much the tour length, the visiting order, and the operator's validity change. You get three things at once: a quantified justification for real road data, a demonstration that you understood why an algorithm's assumptions matter, and a result that essentially no other submission will contain.

08 Validation — an unbroken chain NEW · REVISED

Both documents specified the links. Neither checked that they connect.

Brute force is trustworthy to about `n = 12`. TSPLIB's published optima start at `n = 52`. Revision 1 claimed Held–Karp was exact to `n ≈ 20` and B&B beyond — but between 13 and 51 nothing external checks either of them. That band is exactly where the interesting exact-solver results live, so it cannot be a gap.

It closes with **mutual agreement**: three independently-written exact solvers agreeing on the same instance is strong evidence, because they share no code path. Held–Karp is a dynamic program over subsets; B&B is a tree search with a bound; brute force is exhaustive enumeration. A common bug producing identical wrong answers across all three is vanishingly unlikely.



Every problem size has something checking it. The bands deliberately overlap at n=12 and n=52 rather than merely touching, so each handover is itself verified. The two full-width rules are the checks that apply everywhere: the feasibility gate on all solutions, and the bound-validity invariant that protects the exact solvers from the silent failure in §06.

TIER	CHECK	COVERS
1	Brute force on random instances	n ≤ 12 — the root of trust
2	Brute force ≡ Held–Karp ≡ B&B (every bound variant)	n ≤ 12 overlap, then 13–51 by mutual agreement
3	TSPLIB loader reproduces published tour costs exactly	n ≥ 52, external ground truth
4	Independent feasibility checker on every heuristic output	All sizes, all constraints
5	OR-Tools stored separately with explicit provenance	Reference only — never truth
6	End-to-end real-road run: raw stops → matrix → plan → map	The integration path

> THE TSPLIB ROUNDING TRAP
>
> TSPLIB defines specific distance conventions — `EUC_2D` rounds to the nearest integer, `ATT` and `GEO` use their own formulas. Get the convention wrong and your tour costs differ from the published optimum by a percent or two, which looks exactly like a mediocre solver. Tier 3 exists to catch this *before* any gap number is computed: load a published optimal tour, score it, and require an exact match.

---

09 Evidence discipline NEW

The benchmark is the product. These are the rules that make its numbers mean something.

The research question

> PRIMARY QUESTION
>
> At what problem size and compute budget does exact optimisation stop being worth its cost — and how much solution quality is given up by switching to scalable heuristics?

Every experiment below answers part of this. Having one question makes the report an argument rather than a catalogue.

Experiment families

- **Exact crossover sweep.** Increase `n` until each exact method hits its time or memory budget. Record solved fraction, wall time, peak memory, and proof status. This locates the wall — the central result.
- **Bound quality.** Identical instance, incumbent, branching rule, and strategy; only the lower-bound function changes. Compare nodes explored and pruned, plus bound computation time.
- **Heuristic gap.** Where the optimum is known, repeated seeded instances; report the gap distribution with error bars.
- **Anytime quality.** Large instance; plot incumbent quality against elapsed time for the metaheuristic.
- **Road-matrix effect.** Same stops under Haversine and under OSRM. Quantify how much asymmetry changes tour cost, visiting order, and operator validity (§07).

- **Feasibility engine.** Naive downstream recomputation versus forward-slack acceleration, on identical accepted moves.

Statistics that suit the measurement

- **Runtime: median and IQR**, optionally p90. Runtime distributions are right-skewed by garbage collection, cache effects, and OS scheduling; a mean is misleading.
- **Quality gap: mean with confidence interval** across repeated instances, showing the median too when outliers are visible.
- **Exact-solver success: solved / attempted** under a fixed budget — not just statistics over the runs that happened to finish, which silently discards the hardest instances.
- **B&B: node counts and pruning ratio** alongside time, since time is contaminated by implementation overhead in a way node counts are not.
- Every plot names CPU, RAM, OS, interpreter version, seed policy, and time budget.

Experiment manifests

Every benchmark batch writes a machine-readable manifest: input hashes, matrix hash, code revision, solver configuration, seed, environment, and budget. A figure that cannot be traced to a manifest does not go in the report. This is what makes "one command regenerates everything" true rather than aspirational.

Invariant tests

Property tests over randomly generated instances, run on every commit:

- **A lower bound never exceeds the true optimum** on any brute-force-verifiable instance. The most important test in the project — it is the only thing standing between you and a confidently wrong "proven optimal".
- Route cost is unchanged by serialising and deserialising an instance.
- Every customer appears exactly once in a feasible solution, unless explicitly recorded as unserved.
- No accepted local-search move ever worsens feasibility.
- Identical matrix + seed + configuration produces an identical objective and result trace wherever determinism is promised.
- Tour cost is invariant under rotation of the tour, and under reversal *only* when the matrix is symmetric — a direct test of the §07 issue.

Failure corpus

When a random test finds a counterexample, persist that instance as a named regression fixture rather than trusting the seed to reproduce it. In optimisation software the most valuable tests are the small ugly instances that once broke an invariant — a three-stop instance with a zero-length edge, a time window that opens before the vehicle can physically arrive, two customers at identical coordinates.

---

10 CVRPTW scope control REVISED

The extension that makes it a courier system, kept from becoming a second research project.

CVRPTW sits *on top of* a validated TSP/VRP core. It is not a second exact-optimisation effort, and the system never claims global CVRPTW optimality.

MANDATORY

- Multiple vehicles with capacity.
- **Hard time windows first.** Soft windows only after hard-window semantics are tested — soft windows introduce a penalty weight, and a penalty weight interacts with the objective in ways that are hard to debug on top of untested window logic.
- Service time and route/shift duration.
- Asymmetric travel matrix, with Or-opt-family operators (§07).
- Independent feasibility gate covering every customer exactly once unless explicitly marked unserved.
- Per-stop ETA reconstruction from the exact same matrix the solver used.

SIMPLIFIED FOR THE GRADED BASELINE

- **One capacity dimension: parcel count.** The review recommended picking one; parcel count is the better pick over weight because a viewer can verify a violation by counting pins on the map. Volume and weight become multi-dimensional extensions only if time remains.
- Priority tiers and mandated driver breaks are stretch unless the course rubric names them.
- Static OSRM travel times are enough. Time-dependent traffic matrices are future work.
- Dynamic replanning freezes completed stops and is demo-only, attempted only once the core benchmarks are complete.

NO FALSE OPTIMAL BADGE

For the large courier demo the correct success condition is: a **feasible** plan, a reproducible objective, clear solver provenance, and a measured gap to a named reference where one exists. Global CVRPTW optimality is not required, not claimed, and not implied by any wording in the interface.

---

11 Build plan REVISED

Nine phases. Each has an exit gate — a falsifiable condition, not a description of intent.

> Effort split: **70% core and benchmark evidence · 20% real-road and CVRPTW integration · 10% API and interface.** Deliberately asymmetric. A polished interface cannot compensate for an unvalidated exact solver.

PHASE 0

Contracts and truth

Data model, objective, the status enum and `SolveReport` schema, the independent feasibility checker, the TSPLIB loader, the random generator, and the `core/` import-purity test. Integer seconds and integer metres throughout — float comparison in feasibility checks produces bugs you will not find.

EXIT GATE

A published TSPLIB tour scores to its exact published cost, brute force works, and the purity test fails on a deliberately added `import requests`.

PHASE 1

Distance contract and adapters

Protocol and matrix hashing in `core/`; Haversine, OSRM client, and persistent cache in `adapters/`. Geocoding for CSV ingest. The `is_symmetric` flag is set here and is load-bearing from §07 onward.

EXIT GATE

The same instance runs offline (Haversine) and on OSRM; both matrices log a hash, and the asymmetry flag is correct for each.

PHASE 2

Assignment-complete solver core

Nearest neighbour, Clarke–Wright savings, brute force, Held–Karp, and B&B with the reduced-cost bound. **The assignment's three named techniques are satisfied at the end of this phase.**

> **EXIT GATE**
> All three exact solvers return identical objectives across a broad seeded small-instance suite. Any disagreement blocks Phase 3.

PHASE 3

The Branch & Bound experiment

Hungarian bound, then 1-tree with subgradient ascent gated on the bound-validity invariant. Full instrumentation: nodes created/explored/pruned, peak frontier, incumbent trace, bound time recorded separately. DFS and best-first both available, one fixed as default.

> **EXIT GATE**
> The bound-comparison plot generates from raw benchmark data, and the validity invariant passes on 1,000 random instances for every bound.

PHASE 4

Heuristic quality ladder

Or-opt and relocate first (asymmetry-safe), then 2-opt with its symmetry guard, then simulated annealing. Neighbour lists and don't-look bits — without them local search is unusable past a few hundred stops.

> **EXIT GATE**
> Gap-versus-n and anytime curves reproduce from one command, with error bars from repeated seeded runs.

PHASE 5

CVRPTW constraints

Capacity (parcel count), hard time windows, service time, shift limits, multi-vehicle throughout. Forward time slack for `O(1)` move feasibility. Every operator becomes constraint-aware.

> **EXIT GATE**
> A 150-stop, 6-vehicle instance with windows passes the independent checker and reports `HEURISTIC_FEASIBLE_CVRPTW` — never "optimal".

PHASE 6

Evidence freeze

The full sweep runner, manifests, TSPLIB table, OR-Tools reference column, all plots, and the written findings — including results that contradicted expectation. **The grade lives here.**

> **EXIT GATE**
> Every report number regenerates from one command in a clean environment, and every figure traces to a manifest.

PHASE 7

API and map

Thin FastAPI with `/solve`, jobs, and SSE progress. React + MapLibre with real OSRM polylines. The provenance panel showing method, formulation, status, elapsed time, objective units, comparator, and whether the number is proven or reference-relative.

> **EXIT GATE**
> The demo reads real `SolveReport` fields — no algorithm logic duplicated in the API or the frontend.

PHASE 8

Stretch only

Dynamic insertion of new orders, failed-delivery handling, frozen completed stops. Optionally ALNS as a second metaheuristic, compared against SA.

> **EXIT GATE**
> Attempted only if Phase 6 is complete and stable.

Stop rules

Conditions under which you stop adding and start fixing:

- Do not add a solver while an existing exact solver disagrees with brute force.
- Do not start frontend polish before the benchmark plots regenerate from a clean run.
- Do not implement Phase 8 while the report still contains expected numbers instead of measured ones.
- Do not optimise for speed before the counters that explain where time goes exist.
- Do not compare solvers that saw different distance matrices or different constraint semantics — the matrix hash makes this checkable.

---

12 Definition of done REVISED

Every item verifiable by someone else, on their machine, from your repository.

AREA	ACCEPTANCE TEST
Exact correctness	Every exact solver agrees with brute force across a large seeded small-instance corpus; unsupported cases explicitly excluded.
Validation continuity	No problem-size band is unchecked: brute force, mutual agreement, and TSPLIB overlap at their boundaries.
Bound validity	The LB ≤ optimum invariant passes for every bound on 1,000 random instances.
Published validation	At least five named TSPLIB instances scored under the correct distance convention and compared to published optima.
B&B evidence	At least two lower bounds compared on nodes explored, nodes pruned, bound cost, runtime, and peak frontier.
Crossover result	The practical exact-method size wall reported as measured, on named hardware, under a stated budget.
Heuristic evidence	Gap-versus-n from repeated instances with uncertainty bars.
Anytime evidence	One metaheuristic produces a quality-versus-time curve on a large instance.
Asymmetry handling	The road-matrix experiment is reported, and 2-opt's symmetry guard is demonstrated to trigger.
Real roads	At least one full run on a self-hosted OSRM asymmetric matrix with real route geometry.
CVRPTW feasibility	A realistic fleet instance passes the independent checker; status is "feasible", never "optimal".
Reproducibility	One command regenerates raw data, tables, and plots from pinned inputs, seeds, and configuration.
Claim discipline	Every displayed gap names its comparator; every "optimal" traces to a valid proof.
Report quality	The findings include at least one result that contradicted an initial expectation, and explain why.

13 The report as an argument NEW

Do not structure the write-up as "here are six algorithms." Structure it as a claim supported by experiments:

1. **Problem mismatch.** TSP is the assignment's kernel; courier routing requires partitioning, capacity, time, and asymmetry.
2. **Truth source.** Exact methods establish what "best" means at sizes where proof is computationally possible — and here is the unbroken chain showing no size went unchecked.
3. **The crossover.** Exact methods become impractical; here is exactly where, on this hardware, under this budget.
4. **The quality ladder.** Constructive heuristics are nearly free; local search buys most of the remaining quality; metaheuristics trade increasing time for shrinking gains.
5. **Bound engineering.** Tighter lower bounds eliminate orders of magnitude of search — but their own computation cost is part of the trade.

6. **Reality check.** Real asymmetric road matrices change what a valid route is, and invalidate a textbook operator. Here is the measurement.
7. **Reproducibility.** Every number is rerunnable and every claim is labelled with its proof or reference status.

> THE STANDARD TO HOLD
>
> The strongest version of this project is not the one with the most algorithms or the prettiest map. It is the one where a sceptical examiner can pick any number in the report, ask "how do you know that," and receive an answer that ends in a command they can run.

Waybill — architecture and build plan, revision 2 · Incorporates the architecture review; departures from it are marked and argued in §05 and §07 · Suggested project name; rename freely.